Rulecam documentation

RuleCam is an automated traffic violation detection system. It uses a React frontend, a Flask backend for database management, a FastAPI backend for YOLOv8 AI processing, and VideoDB for cloud video hosting and AI analysis.

1. STRUCTURAL DOMAINS

Client (React + Vite): Displays the dashboard, streams the webcam, draws bounding boxes on a canvas overlay, and records 10-second violation clips.

Controller (Flask Backend): Routes API requests, manages the SQLite database, and runs background worker threads for heavy tasks.

Inference API (FastAPI): Runs the local yolov8n.pt object detection model and processes video frames.

Cloud AI (VideoDB API): Stores video clips in the cloud, provides HLS video streaming, and generates AI summaries and chat replies.

2. STEP-BY-STEP FLOW

1. **Frame Stream:** The React UI grabs webcam frames, converts them to base64, and sends them to Flask every few hundred milliseconds.
2. **Detection:** Flask forwards the base64 frames to FastAPI. FastAPI runs YOLOv8 and returns object coordinates.
3. **Drawing:** React receives the coordinates and draws colored boxes over the live webcam feed.
4. **Recording:** If a violation is detected locally, React records a 10-second video clip and uploads it to Flask.
5. **Thread Hand-off:** Flask saves the video, starts a background thread to process it, and immediately returns a 202 Accepted status to keep the UI smooth.
6. **Verification:** In the background thread:
   - FastAPI checks the video to isolate the exact violation segment; it is uploaded to VideoDB.
   - VideoDB indexes scenes and its LLM generates a verdict (Confirmed or Rejected).
   - Flask saves the final HLS video link, video ID, and verdict in the SQLite database.
7. **Video Chat:** When a user asks a question about a recorded clip, Flask queries VideoDB's LLM using the video's index and returns the answer.

Part 2: Codebase & Subsystems

1. Frontend (src/ Directory)

src/App.jsx: Main dashboard. Manages webcam streaming, records clips with MediaRecorder, and displays live violation tables and video chat.

src/App.css & src/index.css: Styling sheets. Implements a high-contrast Neo-Brutalist design (thick borders, bold colors, and Quicksand/Inter fonts).

2. Controller (backend/ Directory)

backend/app.py: Flask API server. Manages the background worker, writes to the database, and connects to VideoDB.

backend/database.db: Local SQLite database.

3. Inference Service (hf_api/ Directory)

hf_api/main.py: FastAPI server. Loads the YOLOv8 model, processes base64 frames, and crops violation clips.

Part 3: Core Algorithmic Logic

A. Triple Riding Detection

Triple riding is defined as three or more people on one motorcycle. YOLOv8 detects bounding boxes for person P and motorcycle M. The system calculates the overlap ratio:

$$Overlap\ Ratio = Area(P \cap M) / Area(P)$$

If the $Overlap\ Ratio > 0.4$, the person is marked as riding that motorcycle. If a motorcycle has a rider count $R \geq 3$, a violation is flagged.

B. Traffic Signal Jumping

Signal jumping occurs when a vehicle crosses the stop line during a red light. YOLOv8 detects the traffic light and nearby vehicles. The system crops the traffic light box and converts it to HSV color space to identify the active color:

Red Range: $H \in [0, 10] \cup [170, 180],\ S \in [70, 255],\ V \in [50, 255]$
Yellow Range: $H \in [15, 35],\ S \in [150, 255],\ V \in [150, 255]$
Green Range: $H \in [40, 90],\ S \in [50, 255],\ V \in [50, 255]$

If the signal is Red, the lower edge of the traffic light box is set as the stop line boundary (Y_{light}). If a vehicle's lower boundary exceeds the line, a signal-jumping violation is flagged:

$$(y_{max\_vehicle} / height) > Y_{light}$$

C. VideoDB Verification & Chat

Asynchronous Upload & Verification Lifecycle:

Upload MP4 → FastAPI Crops Clip → Upload to VideoDB → Index Scenes → LLM Verdict → Save to SQLite

LLM Verification Prompt:

> *"Respond with exactly 'VERDICT: CONFIRMED' if a traffic violation is visible, or 'VERDICT: REJECTED' if not. Follow with a one-sentence explanation."*

Part 4: Technical Specifications

Fallback: If indexing fails or times out, the system falls back to semantic search: `video.search("identify traffic violation")`

Chat-with-Video Prompt Template:

> You are a traffic violation analysis AI. Answer the user's question concisely based on these video scene descriptions.
>
> **VIDEO SCENES:** {scene_context}
> **USER QUESTION:** {question}
> *Keep your answer under 200 words.*

SQLite Schema: violations

Column Name	Type	Description
id	INTEGER PRIMARY KEY	Unique database identifier.
timestamp	TEXT	Capture time (YYYY-MM-DD HH:MM:SS).
type	TEXT	Violation type (Signal Jumping or Triple Riding).
video_path	TEXT	Local file path for the recorded clip.
vehicle_type	TEXT	Type of vehicle (e.g., motorcycle, car).
status	TEXT	Verification status
videodb_url	TEXT	VideoDB streaming URL (HLS).
ai_analysis	TEXT	AI verdict and reasoning.
videodb_id	TEXT	VideoDB asset ID used for video chat.

Environment Variables

Root Directory (.env)

```
VITE_BACKEND_URL=http://localhost:5005
```

Backend Directory (backend/.env)

```
VIDEODB_API_KEY=sk-xxxx...
PORT=5005
HF_API_URL=https://xxxx...
```